

6CS005
High Performance Computing
Week 7 Workshop
OpenMP Multithreading

Student Number: 2408869

Name: Narayan Lohani

Group: L6CG15

Table of Contents

Question 1	3
Question 2	10

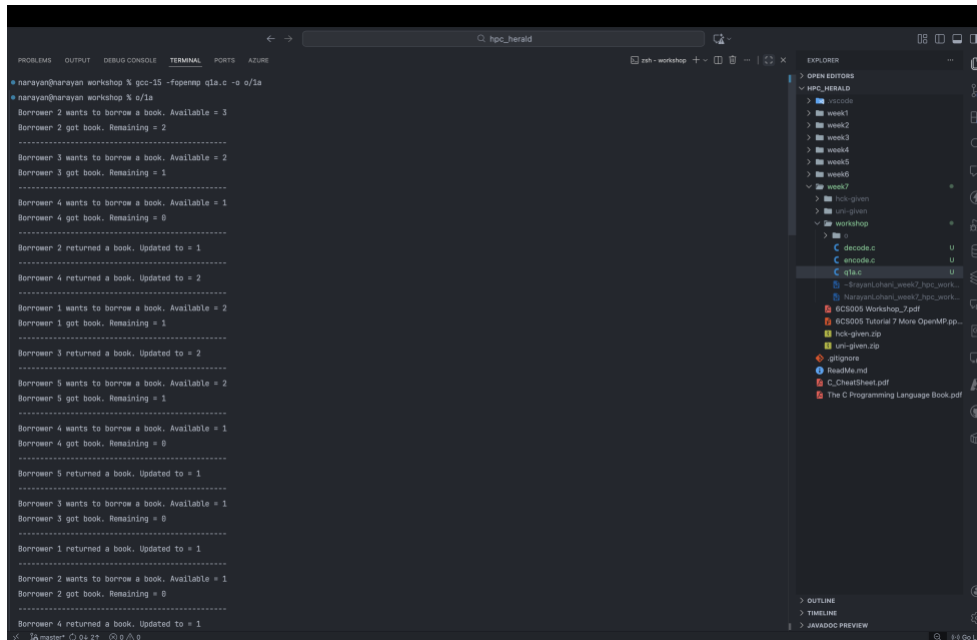
Question 1

The following program uses semaphores to manage the loan of 3 books to a group of 5 borrowers:

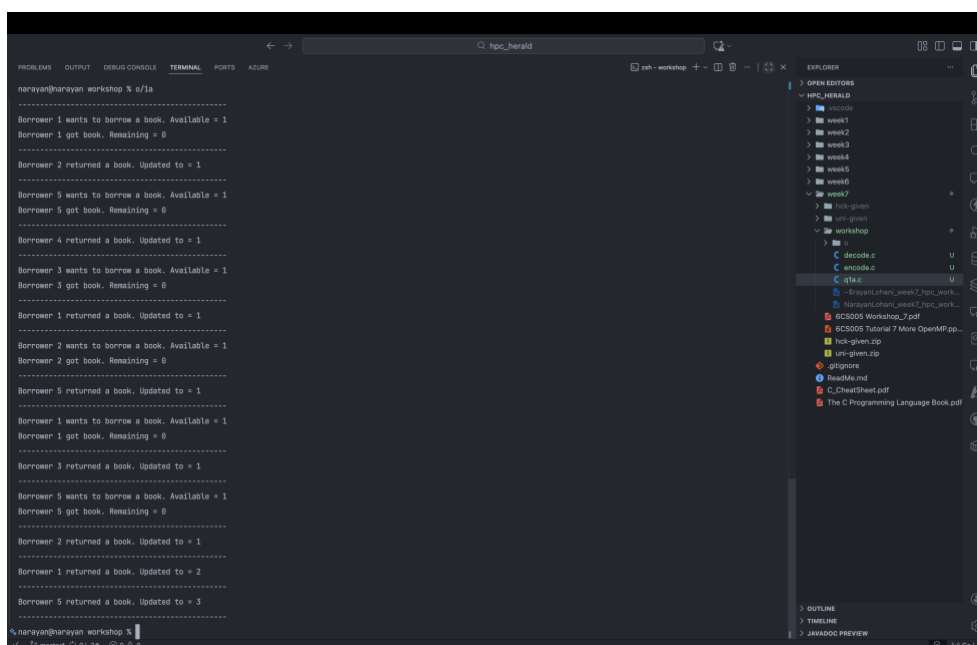
```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>
sem_t semaBooks; /*823
int booksAvailable = 3;
void *borrower(void *p)
{
    for(int i = 0; i<=10; i++){
        printf("Borrower %d wants to borrow a book. ",(int)p);
        printf("Books available = %d\n",booksAvailable);
        sem_wait(&semaBooks); //Borrowing a book
        printf("Borrower %d gets a book. ", (int)p);
        booksAvailable--;
        printf("Books available = %d\n",booksAvailable);
        usleep(10000); //reading book
        printf("Borrower %d is returning a book. ", (int)p);
        booksAvailable++;
        printf("Books available = %d\n",booksAvailable);
        sem_post(&semaBooks); //Returning a book
        usleep(10000);
    }
    return NULL;
}
void main()
{
    pthread_t thrID1, thrID2, thrID3;
    pthread_t thrID4, thrID5, thrID6;
    sem_init(&semaBooks, 0, 3 );
    pthread_create(&thrID1, NULL, borrower, (void *)1);
    pthread_create(&thrID2, NULL, borrower, (void *)2);
    pthread_create(&thrID3, NULL, borrower, (void *)3);
    pthread_create(&thrID4, NULL, borrower, (void *)4);
    pthread_create(&thrID5, NULL, borrower, (void *)5);
    pthread_create(&thrID6, NULL, borrower, (void *)6);
    pthread_join(thrID1, NULL);
    pthread_join(thrID2, NULL);
    pthread_join(thrID3, NULL);
    pthread_join(thrID4, NULL);
    pthread_join(thrID5, NULL);
    pthread_join(thrID6, NULL);
    sem_destroy(&semaBooks);
}
```

- a. Convert it to use OpenMP locks.

To convert this into an OpenMP version, the main change is to replace the semaphore mechanism with OpenMP locks and moving thread creation into OpenMP parallel model. Instead of blocking on a semaphore, each thread repeatedly tries to acquire the lock to check whether a book is free. If a book is available, the critical section updates the booksAvailable, if not, the thread releases the lock and retries after a short sleep. This approach keeps the mutual exclusion behavior but uses OpenMP own synchronization tools.



```
hpc_herald
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
naryan@naryan workshop % gcc-15 -fopenmp q1a.c -o o/1a
naryan@naryan workshop % ./o/1a
Borrower 2 wants to borrow a book. Available = 3
Borrower 2 got book. Remaining = 2
-----
Borrower 3 wants to borrow a book. Available = 2
Borrower 3 got book. Remaining = 1
-----
Borrower 4 wants to borrow a book. Available = 1
Borrower 4 got book. Remaining = 0
-----
Borrower 2 returned a book. Updated = 1
-----
Borrower 4 returned a book. Updated = 2
-----
Borrower 1 wants to borrow a book. Available = 2
Borrower 1 got book. Remaining = 1
-----
Borrower 3 returned a book. Updated = 2
-----
Borrower 5 wants to borrow a book. Available = 2
Borrower 5 got book. Remaining = 1
-----
Borrower 4 wants to borrow a book. Available = 1
Borrower 4 got book. Remaining = 0
-----
Borrower 5 returned a book. Updated = 1
-----
Borrower 3 wants to borrow a book. Available = 1
Borrower 3 got book. Remaining = 0
-----
Borrower 1 returned a book. Updated = 1
-----
Borrower 2 wants to borrow a book. Available = 1
Borrower 2 got book. Remaining = 0
-----
Borrower 4 returned a book. Updated = 1
-----
Borrower 1 returned a book. Updated = 1
-----
Borrower 2 wants to borrow a book. Available = 1
Borrower 2 got book. Remaining = 0
-----
Borrower 4 returned a book. Updated = 1
-----
Borrower 5 returned a book. Updated = 1
-----
Borrower 1 wants to borrow a book. Available = 1
Borrower 1 got book. Remaining = 0
-----
Borrower 3 returned a book. Updated = 1
-----
Borrower 2 wants to borrow a book. Available = 1
Borrower 2 got book. Remaining = 0
-----
Borrower 5 returned a book. Updated = 1
-----
Borrower 1 returned a book. Updated = 2
-----
Borrower 5 returned a book. Updated = 3
-----
naryan@naryan workshop %
```



```
hpc_herald
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS AZURE
naryan@naryan workshop % ./o/1a
-----
Borrower 1 wants to borrow a book. Available = 1
Borrower 1 got book. Remaining = 0
-----
Borrower 2 returned a book. Updated = 1
-----
Borrower 5 wants to borrow a book. Available = 1
Borrower 5 got book. Remaining = 0
-----
Borrower 4 returned a book. Updated = 1
-----
Borrower 3 wants to borrow a book. Available = 1
Borrower 3 got book. Remaining = 0
-----
Borrower 1 returned a book. Updated = 1
-----
Borrower 2 wants to borrow a book. Available = 1
Borrower 2 got book. Remaining = 0
-----
Borrower 5 returned a book. Updated = 1
-----
Borrower 1 wants to borrow a book. Available = 1
Borrower 1 got book. Remaining = 0
-----
Borrower 3 returned a book. Updated = 1
-----
Borrower 5 wants to borrow a book. Available = 1
Borrower 5 got book. Remaining = 0
-----
Borrower 2 returned a book. Updated = 1
-----
Borrower 1 returned a book. Updated = 2
-----
Borrower 5 returned a book. Updated = 3
-----
naryan@naryan workshop %
```

Source Code

```
hpc_herald - q1a.c

1  #include <stdio.h>
2  #include <omp.h>
3  #include <unistd.h>
4
5  int booksAvailable = 3;
6  omp_lock_t lock1;
7
8  void borrower(int);
9
10 int main() {
11     omp_init_lock(&lock1);
12     #pragma omp parallel num_threads(5)
13     {
14         borrower(omp_get_thread_num() + 1);
15     }
16     omp_destroy_lock(&lock1);
17     return 0;
18 }
19
```

```

hpc_herald - q1a.c

20 void borrower(int id) {
21     for (int i = 0; i < 5; i++) {
22         while (1) {
23             omp_set_lock(&lock1);
24
25             if (booksAvailable > 0) {
26                 // take book
27                 printf("Borrower %d wants to borrow a book. Available = %d\n", id, booksAvailable);
28                 booksAvailable--;
29                 printf("Borrower %d got book. Remaining = %d\n", id, booksAvailable);
30                 printf("-----\n");
31                 omp_unset_lock(&lock1);
32                 break; // Exit when borrower gets the book
33             }
34
35             // no books → wait
36             omp_unset_lock(&lock1);
37             usleep(1000); // small wait before retry
38         }
39
40         // read book outside lock
41         usleep(10000);
42
43         // return book
44         omp_set_lock(&lock1);
45         booksAvailable++;
46         printf("Borrower %d returned a book. Updated to = %d\n", id, booksAvailable);
47         printf("-----\n");
48         omp_unset_lock(&lock1);
49
50         usleep(5000);
51     }
52 }
53

```

- b. Modify the program so that it doesn't use locks.

To make the program lock-free, we remove the explicit OpenMP lock and instead rely on OpenMP critical constructs to safely update the shared `booksAvailable` variable without fully blocking threads. Instead of one thread owning a lock and excluding others from the entire borrow-check process, each thread quickly checks availability using a lightweight critical section just large enough to update the shared count. The thread loops until it succeeds in decreasing the count, simulating a non-blocking retry model rather than waiting on a lock. Returning the book uses the same small critical region to increment the counter. Since OpenMP manages these synchronization primitives internally, the code avoids manual lock setup and teardown while keeping the shared variable consistent.

However, going lock-free introduces some trade-offs. The biggest one is fairness: threads may continuously retry and one thread could grab books more often than others, since there's no strict ordering like with a lock. The busy-retry loop also burns a little CPU time, especially under heavy contention, and can cause more context-switch overhead compared to a lock that sleeps waiting threads. Output also becomes harder to keep clean because printing cannot be made fully atomic without reintroducing larger critical regions. In short, the lock-free approach gives more concurrency and avoids lock bottlenecks, but it trades away strict thread ordering, predictable timing, and cleaner output.

```
narayan@narayan workshop % o/1b
Borrower 4 wants to borrow a book. Available = 3
Borrower 4 got book. Remaining = 2
-----
Borrower 3 wants to borrow a book. Available = 2
Borrower 3 got book. Remaining = 1
-----
Borrower 1 wants to borrow a book. Available = 1
Borrower 1 got book. Remaining = 0
-----
Borrower 3 returned a book. Updated to = 1
-----
Borrower 5 wants to borrow a book. Available = 1
Borrower 5 got book. Remaining = 0
-----
Borrower 1 returned a book. Updated to = 1
-----
Borrower 4 returned a book. Updated to = 2
-----
Borrower 2 wants to borrow a book. Available = 2
Borrower 2 got book. Remaining = 1
-----
Borrower 5 returned a book. Updated to = 2
-----
Borrower 2 returned a book. Updated to = 3
-----
narayan@narayan workshop %
```



hpc_herald - q1b.c

```
1  #include <stdio.h>
2  #include <omp.h>
3  #include <unistd.h>
4
5  #define total_books 3
6  int booksAvailable = total_books;
7
8  void borrower(int);
9
10 int main() {
11     #pragma omp parallel num_threads(5)
12     {
13         borrower(omp_get_thread_num() + 1);
14     }
15     return 0;
16 }
17
```



```

hpc_herald - q1b.c

18 void borrower(int id) {
19     int i;
20     for (i = 0; i < 5; i++) {
21         while (1) {
22             int book_borrowed = 0;
23             #pragma omp critical(books)
24             {
25                 if (booksAvailable > 0) {
26                     printf("Borrower %d wants to borrow a book. Available = %d\n", id, booksAvailable);
27                     booksAvailable--;
28                     printf("Borrower %d got book. Remaining = %d\n", id, booksAvailable);
29                     printf("-----\n");
30                     book_borrowed = 1;
31                 }
32             }
33             usleep(1000);
34             if (book_borrowed == 1) break;
35         }
36
37         #pragma omp critical(books)
38         {
39             booksAvailable++;
40             printf("Borrower %d returned a book. Updated to = %d\n", id, booksAvailable);
41             printf("-----\n");
42         }
43         usleep(5000);
44     }
45 }
46

```

Question 2

Following is a C program to encode a three lower case letters into numeric code and decode it back into letters.

encode.c

```
#include <stdio.h>
long encode(char *s)
{
    long a,b,c,x;
    a = s[0];
    b = s[1];
    c = s[2];
    x = (((a*69)+c)*137)+b)*39;
    x = x % 54321;
    return x;
}
void main()
{
    char s[100];
    long x;
    printf("Enter 3 lowercase letters: ");
    scanf("%s",s);
    s[3]='\0';
    x=encode(s);
    printf("Code for %s is %ld\n",s,x);
}
```

decode.c

```
#include <stdio.h>
#include <stdlib.h>
long encode(char *s)
{
    long a,b,c,x;
    a = s[0];
    b = s[1];
    c = s[2];
    x = (((a*69)+c)*137)+b)*39;
    x = x % 54321;
    return x;
}
void main()
{
    char s[4];
    long x,y;
    int i,j,k;
    printf("Enter the code: ");
    scanf("%ld",&x);
    s[3]='\0';
    for(i=0;i<26;i++){
        s[0]=i+'a';
        for(j=0;j<26;j++){
            s[1]=j+'a';
            for(k=0;k<26;k++){
                s[2]=k+'a';
                y=encode(s);
                if(x==y){
                    printf("The letters for code %ld are %s\n",y,s);
                    exit(0);
                }
            }
        }
    }
}
```

- a. Build and run both encode and decode program and verify it successfully decodes the encoded letters.

The given program indeed works as intended.

```
● narayan@narayan workshop % gcc encode.c -o o/en
● narayan@narayan workshop % o/en
Enter 3 lowercase letters: kai
Code for kai is 31911
● narayan@narayan workshop % gcc decode.c -o o/dec
● narayan@narayan workshop % o/dec
Enter the code: 31911
The letters for code 31911 are kai
❖ narayan@narayan workshop %
```

- b. Modify the program to use the OpenMP for loop “collapse” method to spread the work across 16 threads.

```
• narayan@narayan workshop % gcc-15 -fopenmp decode.c -o o/dec
• narayan@narayan workshop % o/dec
Enter the code: 31911
The letters for code 31911 are kai
❖ narayan@narayan workshop %
```

```
hpc_herald - decode.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4
5  long encode(char* s) {
6      long a, b, c, x;
7      a = s[0];
8      b = s[1];
9      c = s[2];
10     x = (((a * 69) + c) * 137) + b) * 39;
11     x = x % 54321;
12     return x;
13 }
14
15 int main() {
16     long x, y;
17     char result[4] = { 0 };
18     int found = 0;
19
20     printf("Enter the code: ");
21     scanf("%ld", &x);
22
```

```

hpc_herald - decode.c

23 #pragma omp parallel for collapse(3) num_threads(16) shared(found, result)
24     for (int i = 0; i < 26; i++) {
25         for (int j = 0; j < 26; j++) {
26             for (int k = 0; k < 26; k++) {
27                 // if some thread already found the answer
28                 if (found) continue;
29                 char s[4];
30                 s[0] = 'a' + i;
31                 s[1] = 'a' + j;
32                 s[2] = 'a' + k;
33                 s[3] = '\0';
34
35                 long y = encode(s);
36                 if (y == x) {
37 #pragma omp critical
38                     {
39                         if (!found) {
40                             found = 1;
41                             result[0] = s[0];
42                             result[1] = s[1];
43                             result[2] = s[2];
44                             result[3] = '\0';
45                         }
46                     }
47                 }
48             }
49         }
50     }
51     if (found)
52         printf("The letters for code %ld are %s\n", x, result);
53     else
54         printf("No match found.\n");
55     return 0;
56 }
57

```